Kinvecta

TRAIN-THE-TRAINER SERIES · DEEP DIVE B

Containers, scanning and the CI/CD pipeline

One box, scanned once, signed, and trusted everywhere it runs.



20-minute video

Stages 2–5

DevSecOps for AI workloads

Kinvecta

TODAY

By the end, your learners can…

1
Explain DevSecOps, images and where containers run

2
Read a hardened Dockerfile and a scan report

3
Walk the CI/CD line and its gates

4
Set default-deny network rules, both directions

CHAPTERS · 20 MIN

drawn to scale



Kinvecta

WHY THIS MATTERS

Three problems a good pipeline solves

01

“It worked on my laptop”

The agent behaves differently in staging, and nobody can say exactly what shipped.

02

“40 HIGH findings. Now what?”

Scanners shout. Teams fix the wrong things, or switch the scanner off.

03

“The scanner was the attack”

March 2026: a hijacked scanner Action stole CI secrets from the pipelines that ran it.

The question for this deep dive:

How do we ship one box we can trust, and keep trusting?

Kinvecta

DEVSECOPS

Security in every stage, not one gate at the end

Shift left

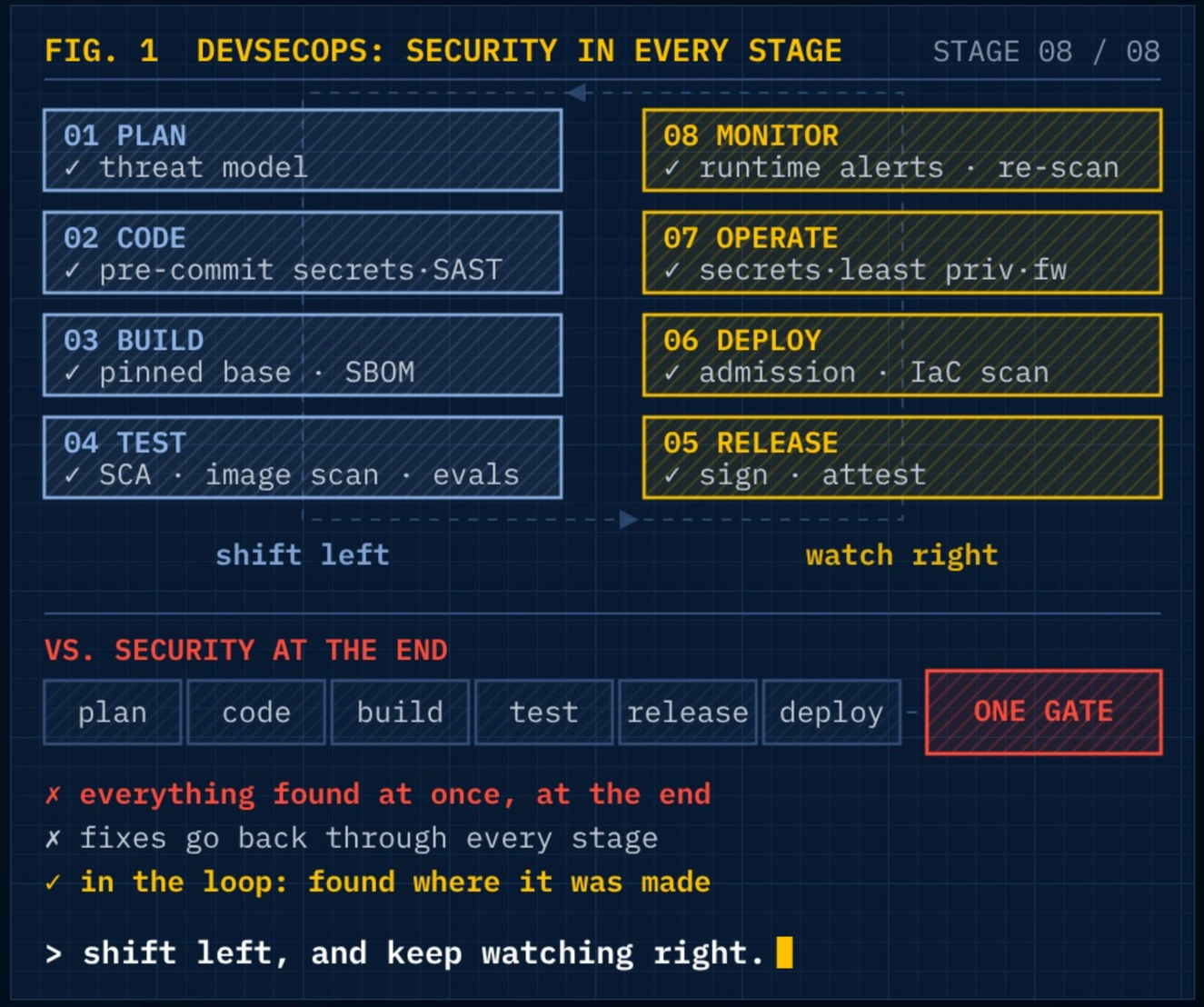
Threat model, secret and code scans while the change is small and cheap to fix

Gate the release

Scan, sign and attest before anything ships

Keep watching right

Runtime alerts, rescans and least privilege after it's live



Kinvecta

CONTAINERS EXPLAINED

One box, built once, run anywhere

Image

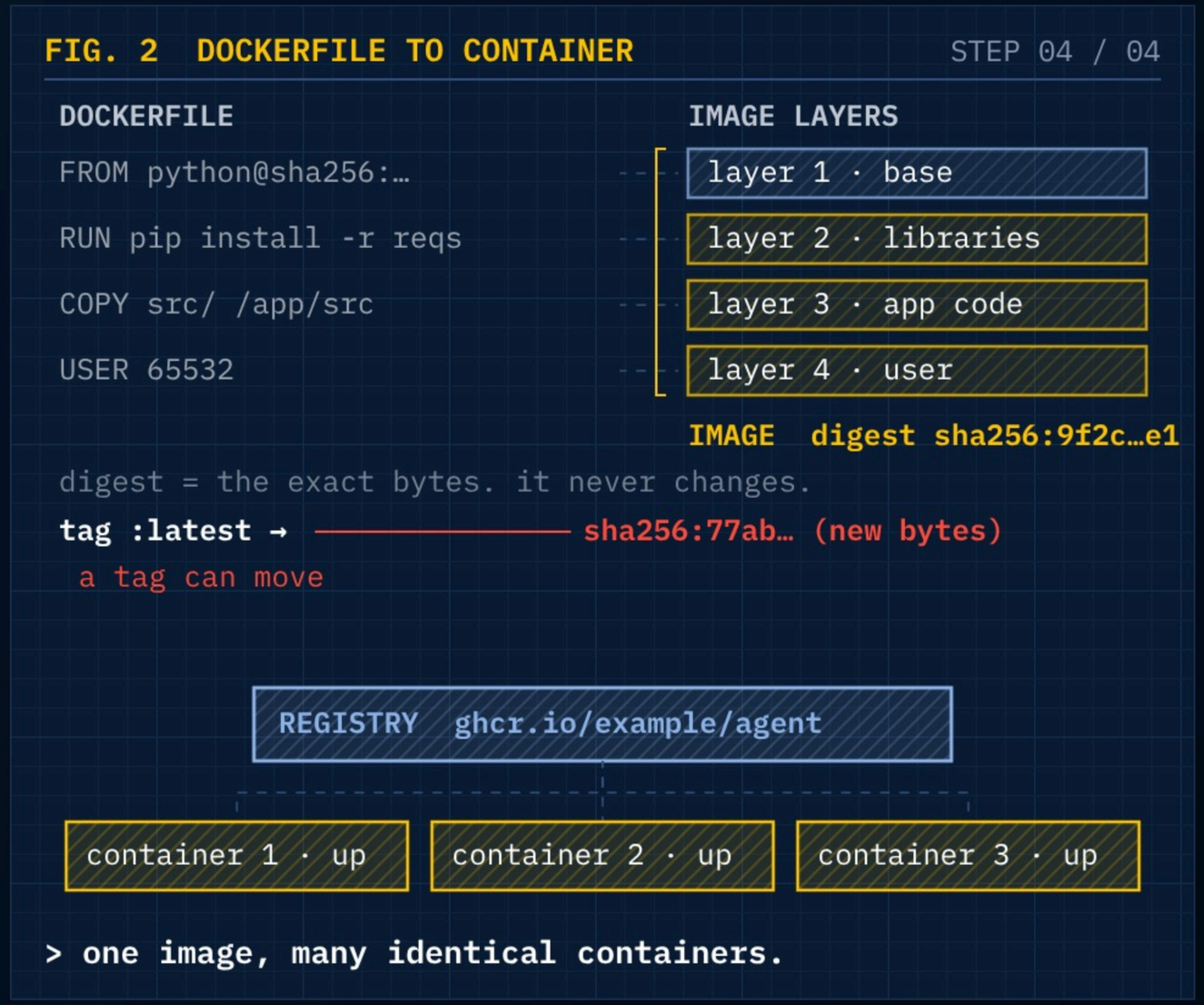
The sealed box: your code, the runtime and libraries. It never changes.

Container

The box opened and running. Many can run from one image.

Registry and digest

The warehouse. The digest (sha256) is the box's true identity; a tag can move.



Kinvecta

WHERE CONTAINERS RUN

Same box, different ships

Same digest everywhere
Laptop, server, serverless, cluster or edge

Who patches what changes
Managed services patch more; your image, code and keys stay yours

Untrusted code needs a sandbox
gVisor, Firecracker or Kata



Kinvecta

HARDENING THE IMAGE

Pack only what the app needs

Minimal base, pinned by digest
No shell, no package manager; the exact base you scanned

Multi-stage build
Compilers, caches and git stay in the build stage and never ship

No secrets, non-root
Keys come at run time; the app runs as an ordinary user

FIG. 4 HARDEN THE DOCKERFILE — FIX 04 / 04

```
FROM python:latest
COPY . /app
RUN pip install -r reqs.txt
ENV API_KEY=sk-live-…
CMD ["python","app.py"]
```

BOX STATS

CVEs	12
size	60 MB
secrets	0
root	no

(illustrative)

```
1  Minimal base, pinned by digest
   FROM …/python@sha256:…

2  Multi-stage: build tools stay behind
   COPY --from=build /app /app

3  No secrets in the image
   inject at run time (rm later leaves it in a layer)

4  Non-root user
   USER 65532
```

> fewer things in the box, fewer things to break.

Kinvecta

READ A HARDENED DOCKERFILE

Two stages, a pinned base, no keys

```
# syntax=docker/dockerfile:1
# build stage: has pip and compilers
FROM cgr.dev/chainguard/python:latest-dev@sha256:<builder> AS build
WORKDIR /app
COPY requirements.txt .
RUN python -m venv /app/venv && \
    /app/venv/bin/pip install --require-hashes -r requirements.txt
COPY src/ ./src/
# runtime stage: no shell, no pip, non-root
FROM cgr.dev/chainguard/python:latest@sha256:<runtime>
COPY --from=build /app/venv /app/venv
COPY --from=build /app/src /app/src
USER 65532
ENTRYPOINT ["python", "-m", "src.server"]
# no API keys here: injected at run time
```

BASE IMAGE OPTIONS

Docker Hardened Images
Free and open source since 17 Dec 2025; SBOM + SLSA provenance

Chainguard
Wolfi-based; free tier is :latest only

Google distroless
Runtime only, no shell; :nonroot variants

Red Hat UBI
RHEL-based; supported on OpenShift

Alpine · Debian-slim
Fine starting points, not hardened by default

Bitnami
Paid since Sep 2025: audit old references

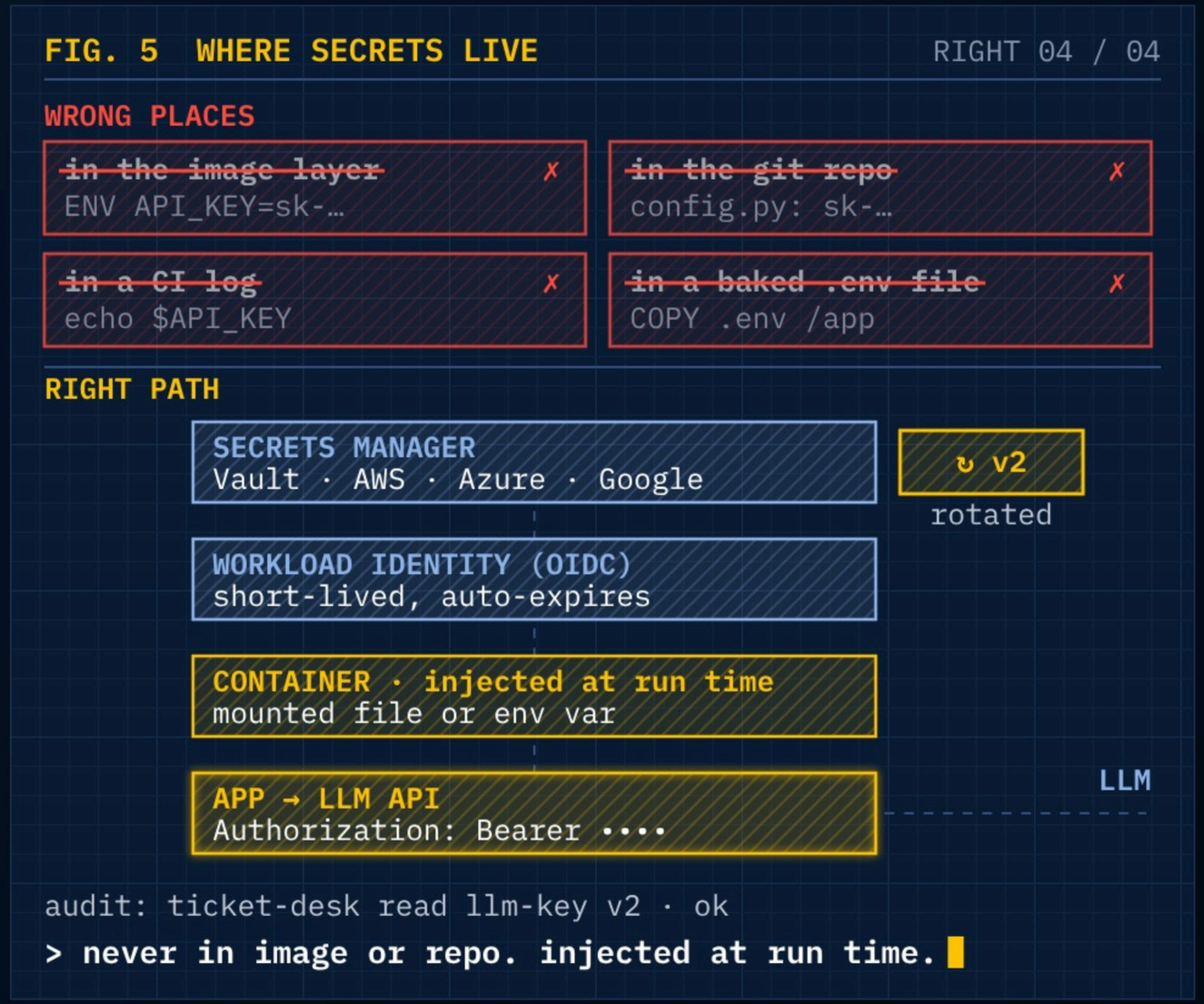
SECRETS

Keys never live in the image or the repo

Layers are permanent
A key deleted in a later step is still in the earlier layer

A manager, not a file
Vault (or OpenBao), AWS Secrets Manager, Azure Key Vault, Google Secret Manager

Short-lived, injected, rotated
Workload identity, injected at run time, rotated on a schedule



Kinvecta

SCANNING

Airport security for software

One threat per machine

Secrets, our code, libraries, config, the image, the running app

Scan early

Commit and pull-request checks are the cheapest place to fix

Scan again later

New CVEs appear after you ship; registries rescan stored images



TOOL REFERENCE · 1 OF 2

Scanners by category

CATEGORY	OPEN SOURCE	COMMERCIAL / HOSTED
Image vulns + SBOM	Trivy · Grype · Syft	Docker Scout · Anchore Enterprise · Snyk Container
Secrets	Gitleaks · TruffleHog · Betterleaks (new, Mar 2026)	GitHub secret scanning + push protection
SAST (our code)	Semgrep CE · CodeQL (public repos) · SonarQube Community Build	Semgrep AppSec · GitHub Code Security · SonarQube Server
SCA + updates	OWASP Dependency-Check · OSV-Scanner · Renovate · Dependabot	Snyk Open Source · Socket
IaC + misconfig	Trivy config (absorbed tfsec) · Checkov · KICS	Cortex Cloud (was Prisma Cloud) · Wiz (now Google)
DAST + malware	ZAP · ClamAV · YARA	Aqua · Cortex Cloud · Wiz

Versions on 20 Oct 2026: Trivy 0.76.0 · Grype 0.121.0 · Syft 1.55.0 · Gitleaks 8.30.1 · Semgrep 1.182.0. Examples, not endorsements.

Kinvecta

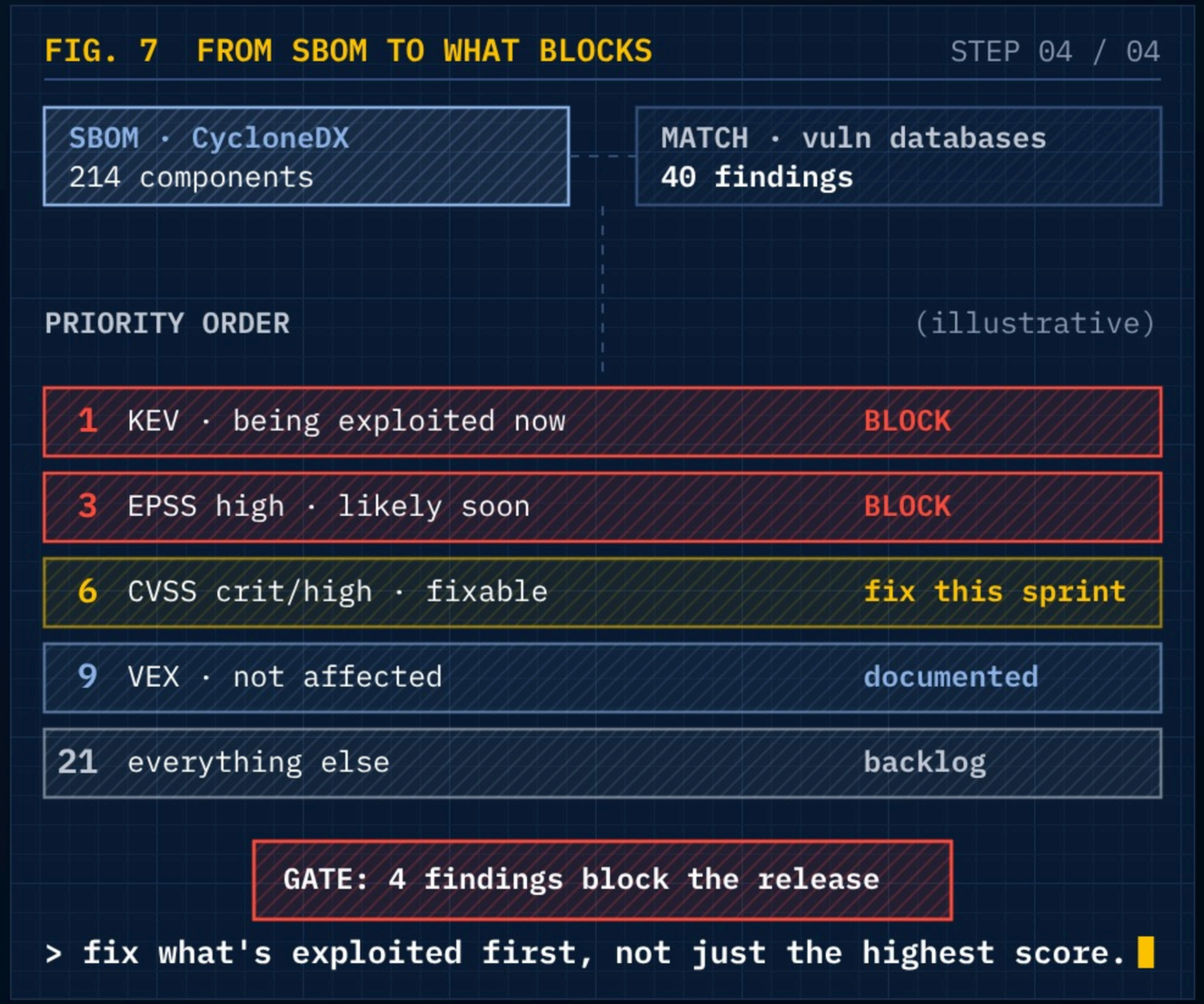
TRIAGE

Fix what is exploited first

CVE · CVSS
The bug's ID, and how bad it could be (0–10)

EPSS · KEV
How likely it's used soon; CISA's list of bugs being exploited now

VEX
A machine-readable note: "not affected, because…", to cut false alarms



Kinvecta

SIGNING AND ADMISSION

Prove it’s the same box we scanned

Sign the digest

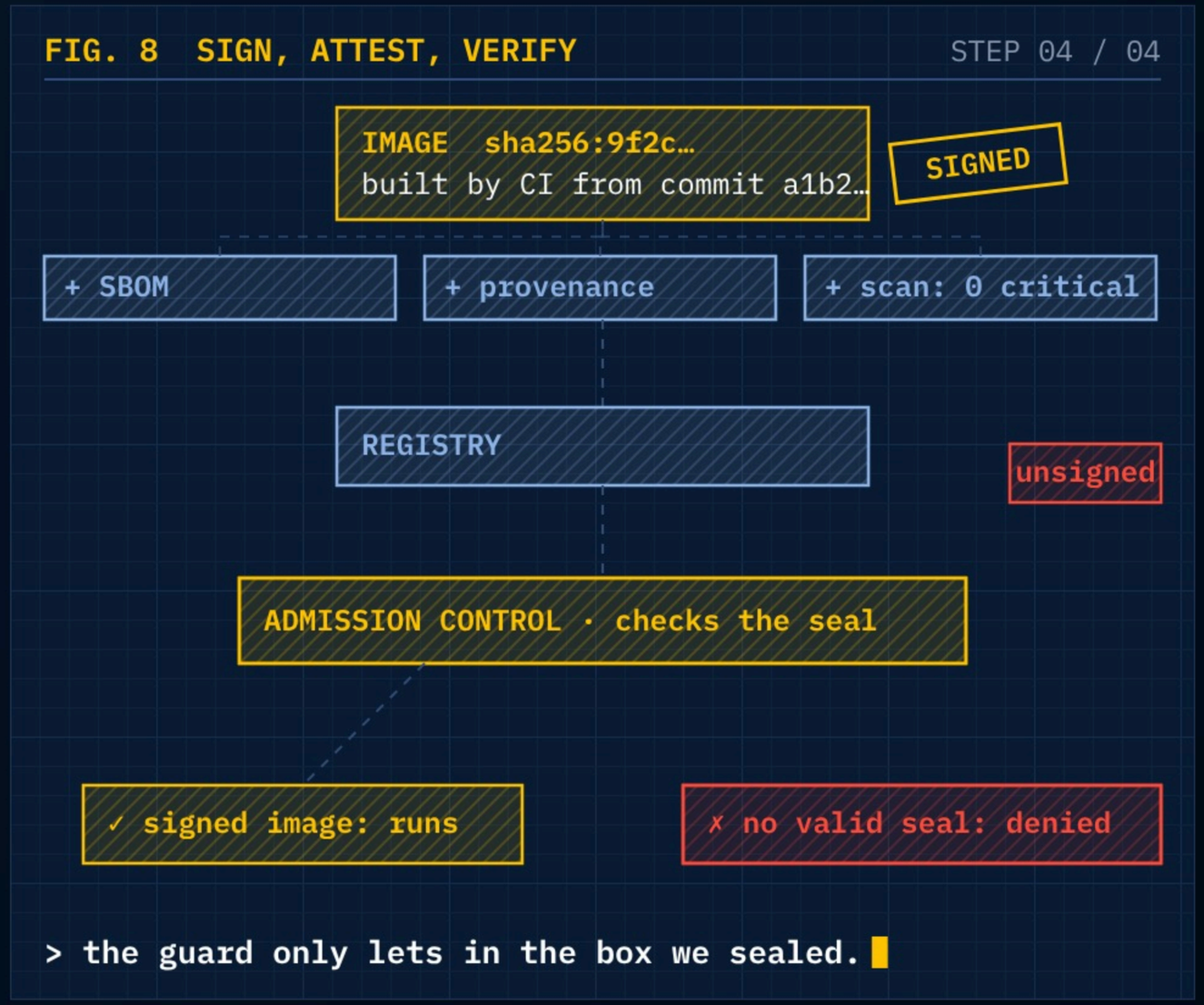
Cosign (keyless: the CI identity) or Notation with your own PKI

Attest

Attach the SBOM, provenance (SLSA) and scan results to the digest

Verify at the door

Admission control refuses anything unsigned or unverified



Kinvecta

TOOL REFERENCE · 2 OF 2

Signing, registries, the line and runtime

CATEGORY	OPEN SOURCE	COMMERCIAL / HOSTED
Signing + provenance	Sigstore Cosign v3 · Notation · SLSA / in-toto	GitHub artifact attestations (actions/attest)
Registries	Harbor (Trivy built in) · Project Quay (Clair)	GHCR · ECR · ACR · Artifact Registry · Artifactory + Xray
Policy + admission	Kyverno (CNCF graduated, Mar 2026) · OPA Gatekeeper · Sigstore policy-controller	Kubernetes ValidatingAdmissionPolicy is built in
CI/CD + GitOps	Jenkins · Tekton + Chains · Argo CD · Flux	GitHub Actions · GitLab CI · Azure DevOps
Secrets	OpenBao · External Secrets Operator · Secrets Store CSI	Vault (IBM) · AWS Secrets Manager · Azure Key Vault · Google Secret Manager
Runtime + monitoring	Falco · Tetragon · OpenTelemetry GenAI conventions (not yet stable)	CNAPP sensors (Aqua, Cortex Cloud, Wiz) · Langfuse · Datadog · Splunk

Versions on 20 Oct 2026: Cosign 3.1.4 · Kyverno 1.19.2 · Argo CD 3.5.6 · Flux 2.9.6. Examples, not endorsements.

Kinvecta

CI/CD

An assembly line with inspection stations

One build, many environments

Staging and prod run the identical digest; never rebuild for prod

Break the build

Fail on fixable critical, any KEV, or secrets in a layer

Time-boxed exceptions

An owner, an expiry and a VEX note, or teams switch the scanner off



Kinvecta

READ THE PIPELINE

A GitHub Actions excerpt, annotated

```
permissions: { contents: read }
steps:
  - uses: actions/checkout@<SHA> # v6
  - run: semgrep scan --config p/owasp-top-ten --error
  - name: Build, load locally (not pushed yet)
  - name: Trivy image scan (the gate)
    uses: aquasecurity/trivy-action@<SHA> # v0.36.0
    with: { severity: "CRITICAL,HIGH",
            ignore-unfixed: true, exit-code: "1" }
  - name: Push (only after the scan passes)
  - run: cosign sign --yes "$IMAGE" # $IMAGE = …@sha256:digest
  - run: cosign attest --yes --type cyclonedx \
         --predicate sbom.cdx.json "$IMAGE"
  - uses: actions/attest@<SHA> # v4 · SLSA provenance
```

1 Least privilege
Jobs opt in to more permissions

2 Pin every action by full SHA
<SHA> = placeholder: fill it from each Releases page, or let Renovate do it

3 Scan before push
exit-code 1 breaks the build

4 Sign the digest
Attach the SBOM and provenance to it

Kinvecta

SUPPLY-CHAIN LESSON

Your scanner is a dependency too

March 2026

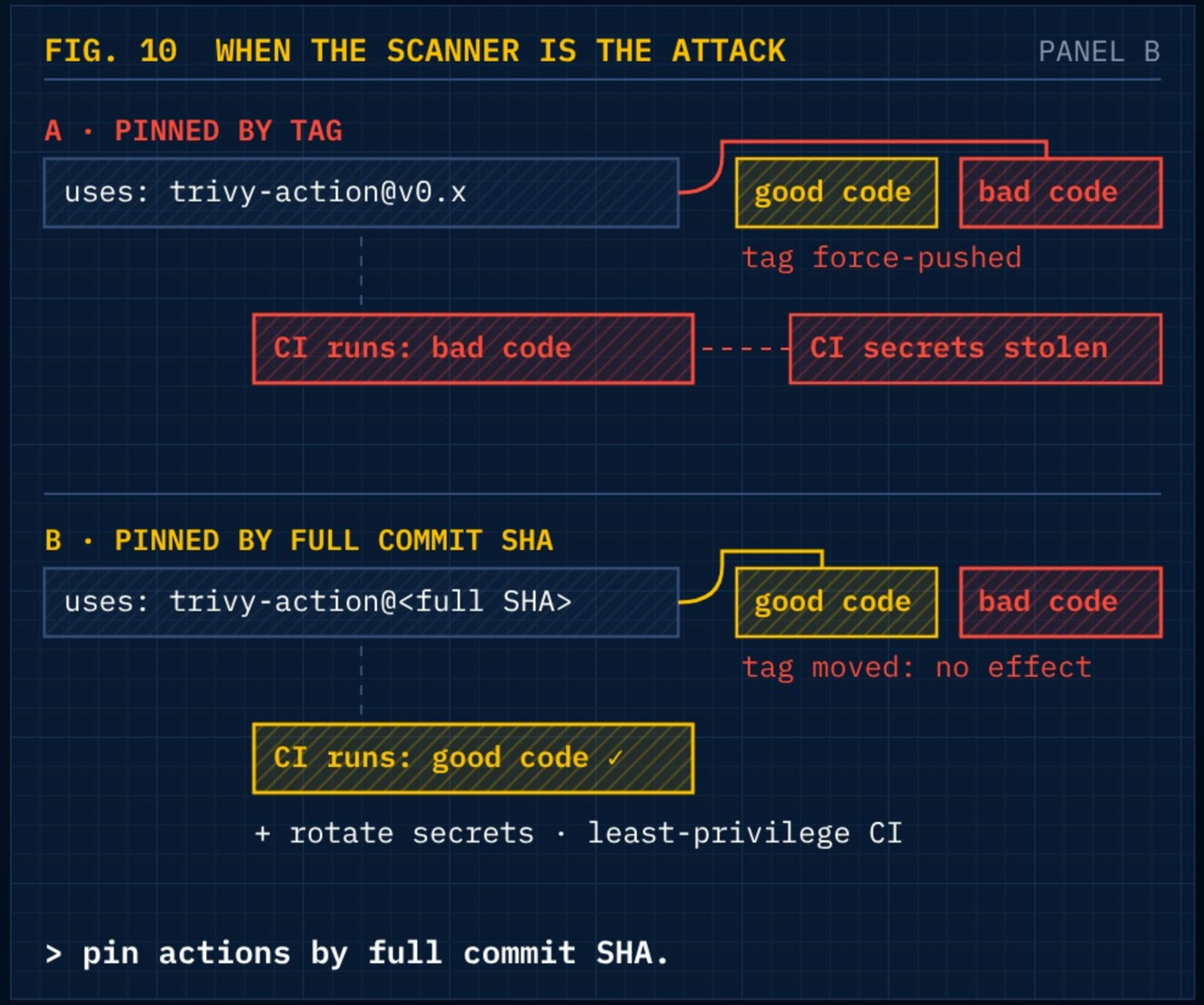
Attackers force-pushed tags of trivy-action and setup-trivy to steal CI secrets

Pin by full commit SHA

Tags move; a SHA can't. GitHub can enforce SHA pinning org-wide.

Least-privilege CI

Short-lived OIDC credentials, minimal permissions, rotate after a compromise



Kinvecta

FIREWALLS

Default deny, both directions

Layers, outside in

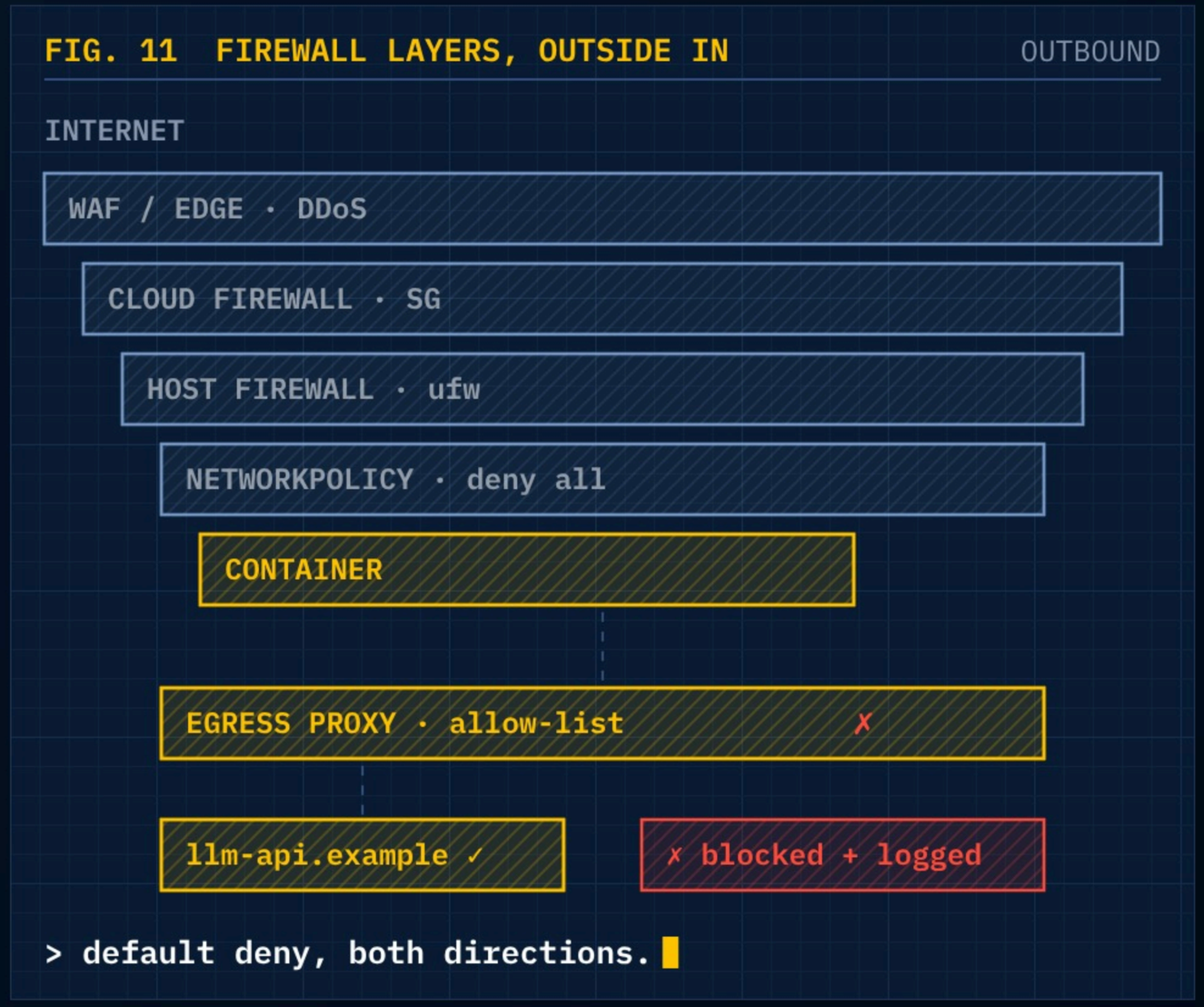
WAF, cloud firewall, host firewall, NetworkPolicy

Check the Docker gotcha

Published ports skip ufw; use DOCKER-USER or bind to localhost

Egress allow-list

AI reaches only the LLM API and approved backends; the rest is logged



Kinvecta

NETWORK CONTROLS · TOOL REFERENCE

Each layer, and the tools that do it

LAYER	WHAT IT DOES	COMMON TOOLS AND SERVICES
1 · Edge: WAF + DDoS	Filters inbound web requests; rate-limit costly LLM endpoints	AWS WAF + Shield · Azure Front Door WAF · Google Cloud Armor · Cloudflare · Akamai
2 · Cloud firewall	Allow or deny by IP, port and protocol at the network	AWS Security Groups + NACLs · Azure NSGs · Google VPC firewall rules
3 · Egress firewall / proxy	One exit with a hostname allow-list and logs	AWS Network Firewall · Azure Firewall · Google Cloud NGFW · Squid · Envoy
4 · Host firewall	Filters on the server; watch Docker's bypass	nftables / iptables · firewalld · ufw with the DOCKER-USER chain
5 · NetworkPolicy	Default-deny between pods; no hostname field	Enforced by the CNI: Calico · Cilium · Antrea
6 · FQDN policy + mesh	Egress by DNS name; mutual TLS between services	Cilium toFQDNs · GKE FQDN policies · Istio · Linkerd

DEFAULT DENY Inbound only from the load balancer. Outbound only to DNS, the LLM API and approved backends; the rest is logged.

AI-SPECIFIC GATES

Model, prompt and tools are build inputs

Pin and version

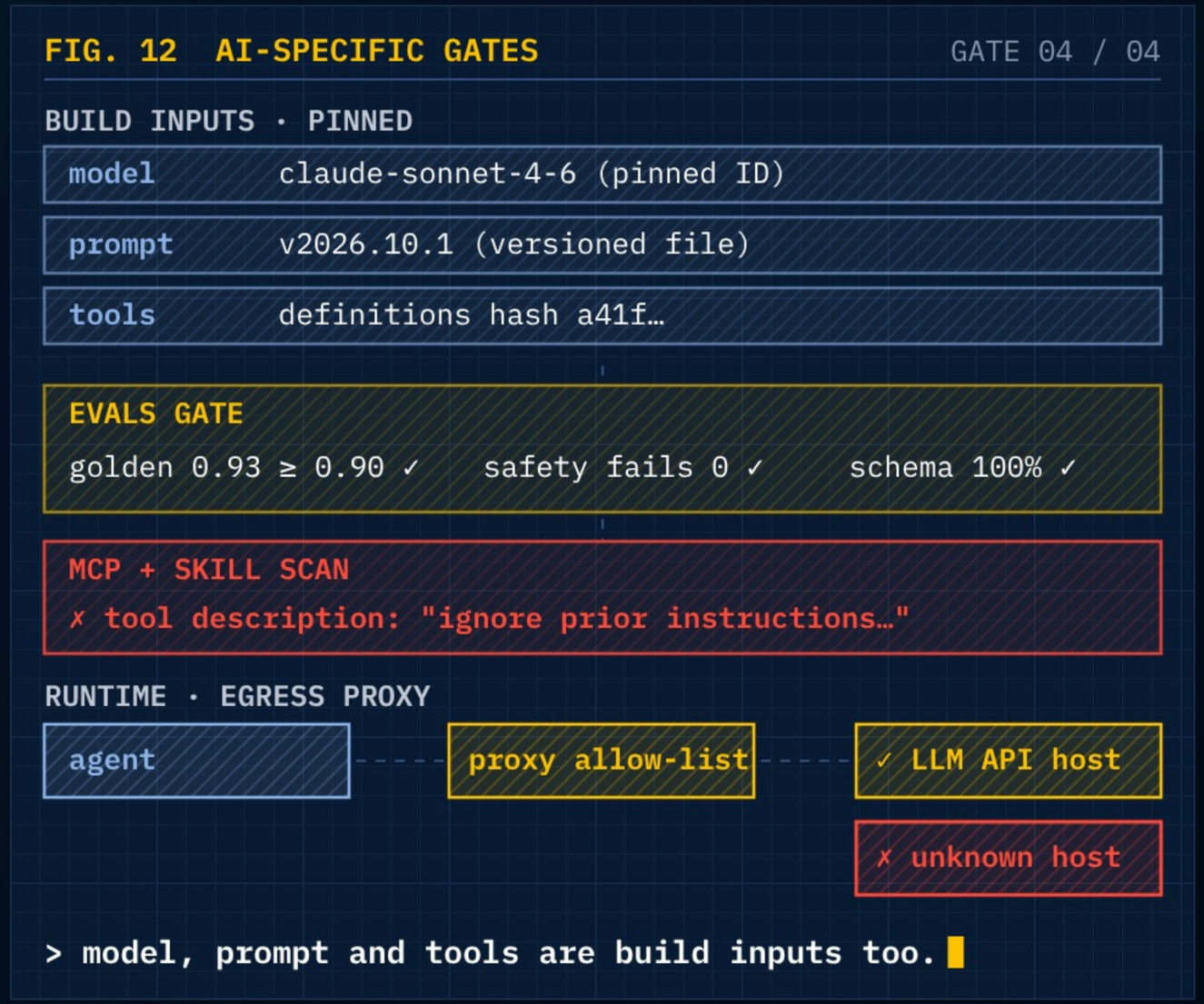
Fixed model ID, prompt files in Git, tool definitions hashed

Evals as a gate

On every change and on a schedule: Promptfoo, garak, Inspect

Scan agents, contain egress

snyk-agent-scan, ModelScan; a proxy allow-lists the LLM API



Kinvecta

THE RULE OF THUMB

Pin everything.
Scan before it ships.
Sign it, and ship the same digest.

One scanned image, signed, promoted unchanged and fenced in both directions, is software you can trust.

Kinvecta

EXERCISE

Find the problems, then triage

Pause & try

A Find 8 problems

```
FROM python:latest
COPY . /app
RUN pip install -r /app/requirements.txt
ENV OPENAI_API_KEY=sk-live-EXAMPLE
RUN rm /app/.env
CMD ["python", "/app/agent.py"]
# .github/workflows/ci.yml
- uses: aquasecurity/trivy-action@master
- run: docker push ghcr.io/acme/agent:latest
- run: trivy image ghcr.io/acme/agent:latest || true
```

B Triage

Policy: block KEV, EPSS ≥ 0.1 and fixable CRITICAL. No fix: time-boxed exception. Not reachable: VEX.

ID	CVSS	EPSS	KEV	FIX	NOTE
A	9.8	0.92	yes	yes	
B	9.1	0.01	no	yes	
C	7.8	0.03	no	no	
D	8.1	0.02	no	yes	code never called
E	5.3	0.01	no	yes	
F	7.5	0.61	no	yes	

Illustrative findings, not real CVEs.

Kinvecta

CHECK FOR UNDERSTANDING

Can every learner answer these?

1

Why sign and deploy by digest instead of tag?

2

A key was added in one step and deleted in the next. Safe to publish?

3

A scan shows 40 HIGH findings. What do you fix first?

4

Scanning vs signing: why do you need both?

5

ufw denies incoming, but docker run -p 8080 is still reachable. Why, and two fixes?

6

A scanner's GitHub Action was hijacked. Which two controls limit the damage?

Kinvecta

DEEP DIVES COMPLETE

Build the server. Box it. Ship the same digest.

FIG. 13 BOTH DEEP DIVES A + B

The two deep dives, together

- Deep Dive A · build an MCP server
- Deep Dive B · containers, scanning, CI/CD
- Before you teach: re-check the dated items